



大连理工大学

DALIAN UNIVERSITY OF TECHNOLOGY

基于 xv6 的内核功能扩展与性能量化评估

OS课程设计：方案 A OS 内核实现

汇报人：袁善

2026-06-22

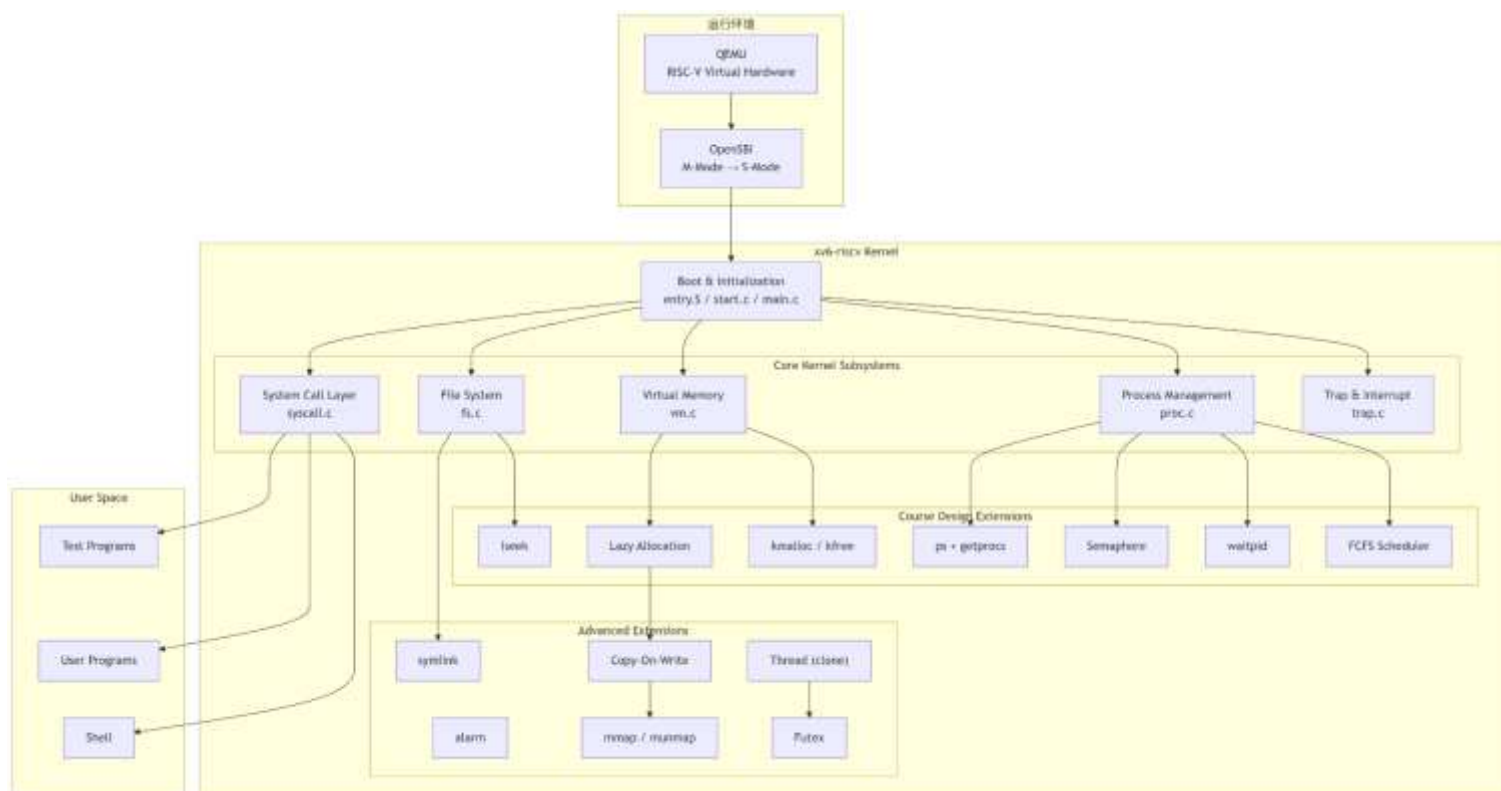
■ 项目摘要

- 本项目**基于 MIT xv6-riscv**，在保持原有体系结构稳定性的前提下，**成功扩展并实现了 16 个新增系统调用**。通过引入内核字节分配器、按需分页、写时复制、VMA 存储映射、FCFS 动态调度切换、轻量级线程、Futex 同步、符号链接等多项现代操作系统关键特性，进一步拓宽了系统的实际应用边界。
- 开发环境：Windows 11 + VSCode (SSH 远程) & Ubuntu 22.04, RISC-V 交叉编译器, QEMU 7.2.0

- **实现**：共支持 **37 个系统调用**（其中**增量实现 16 个**）。包括内核级堆分配器、按需分页、写时复制、文件内存映射、FCFS 与 RR 动态调度切换、轻量级线程、用户态快速同步互斥体以及信号量、异步定时器、软链接等模块。

- **验证**：系统通过**增量的 20 项单元测试**、**xv6 原生 usertests** 集成测试，在 **grind 压力测试** 下持续运行，未发生内核 Panic 或死锁。**达到课程标准**。

- **性能**：移植**极简 Transformer 推理引擎 llama.c** 并加载 stories260K 模型，针对**多核并行能力、同步原语效率及存储映射机制**设计对比实验，量化评估内核相关子系统的实际开销。



■ 项目完成情况

<https://github.com/yuuichi33/OS>



模块	xv6 已实现功能	本项目扩展实现
系统启动	M→S 态切换、内核加载、栈初始化、启动日志	—
中断与异常	时钟中断、键盘输入、ecall 系统调用	缺页异常处理 (Lazy/COW/VMA)、用户态异常分类拦截 (scause 2/13/15 诊断)
内存管理	物理页分配、Sv39 虚拟内存	kmalloc/kmfree 内核堆分配器、按需分页、COW Fork、mmap/munmap
进程管理	PCB、fork/exec、RR 调度	FCFS 非抢占调度 (RR/FCFS 动态切换)、waitpid (WNOHANG)、Semaphore 信号量、Alarm 异步定时器
文件系统	目录文件操作、路径解析、重定向、mkfs	lseek (SEEK_SET/CUR/END)、Symlink 软链接
用户程序	ELF 加载、Shell、管道	ps 命令 (getprocs 系统调用)、异常隔离 (用户态崩溃不 panic)
扩展功能	—	alarm、COW Fork、mmap/munmap、clone 线程、futex 用户态锁

➤ 汇报内容

- 系统设计目标与技术选型
- 系统总体架构
- subsystem设计与实现
 - 虚拟内存管理 (kmalloc、Lazy、COW、mmap)
 - 进程管理 (FCFS/RR调度、waitpid、信号量、Alarm)
 - 文件系统 (lseek、Symlink)
 - 线程与同步机制 (clone、Futex)
 - 中断异常与用户程序 (Trap分发、异常隔离、ps)
- 系统测试与稳定性验证
- 量化性能评估与对比实验
- 创新点、不足及展望

➤ 交付物

- 14 周 进度报告。
- 16 周 结题报告PDF、汇报PPT、源码及文档。

■ 系统设计目标与技术选型

- **设计目标**: 在保持原生 xv6 结构稳定的前提下, 实现**课程设计要求的**操作系统关键机制, 并引入**部分现代 Unix/Linux 内核设计思想**, 提升系统的完整性。
- **开发基准**:
 - 基于 **RISC-V xv6** 内核进行增量式开发, 重点参考 Linux、MIT 6.S081 与 Re-Xvapor。
 - 为保障工具链**兼容性**, 代码回滚至 2022 年底稳定提交 (commit 74c1eba) 作为**基准**。
- **系统引导启动流程简述**:

```
QEMU → OpenSBI (M-mode 初始化与硬件配置)
      → 执行 mret 切换至 S-mode
      → entry.S (各 CPU 核分配独立的内核栈)
      → start.c (配置监听寄存器与中断委托)
      → main.c (主 CPU 初始化内存、进程、文件系统等, 激活次 CPU)
      → scheduler() → 调度 init 进程 → 启动用户态 Shell
```

系统在 **QEMU** 中稳定启动并进入 Shell, 启动过程可复现, 无异常重启。此部分**完全复用 xv6 已实现功能**。

● RISC-V 处理器架构

- **指令设计简洁**: 规避了 x86 繁重的历史兼容包袱, RV64 寄存器与控制状态寄存器 (CSRs) 设计清晰, 极大简化了上下文切换。
- **标准特权级划分**: 具备明确的 U/S/M 三个特权级, 能直观映射进程隔离与内核管理。

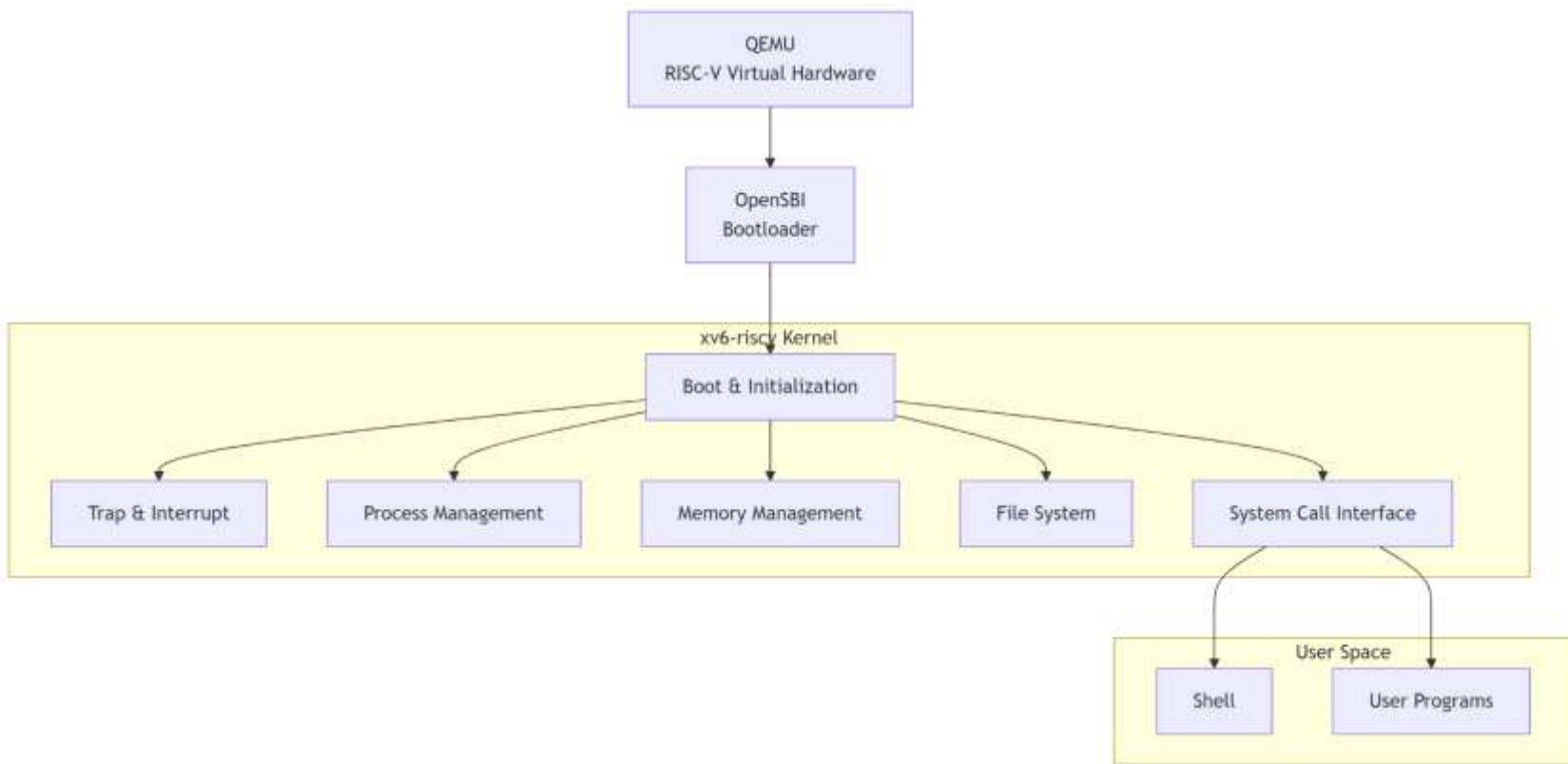
● xv6 内核基准

- **体量精简, 抽象完整**: 相比复杂的 Linux, xv6 保留了核心抽象 (PCB、VM、VFS、Syscall), 代码体量适中, 为后续 COW、Lazy 分配、clone 线程等高阶特性的引入留出了充足的改写与扩展空间。

● QEMU + OpenSBI 模拟平台

- **规范化引导**
- **硬件级页表对齐**
- **并发仿真环境**

■ 系统总体架构



- **QEMU**: 提供 **RISC-V 虚拟硬件环境** (-machine virt, 4 核, 128MB 内存)。
- **OpenSBI**: 完成机器态 (M-Mode) 到监管态 (S-Mode) 的**初始化和特权级切换**。
- **xv6 内核**: 负责 CPU、内存、磁盘、中断等**全部资源的管理和系统服务**。
- **系统调用层**: 提供用户态与内核态的**交互接口** (共 37 个系统调用)。
- **用户空间**: 运行 Shell 和应用程序。

■ 虚拟内存管理

- xv6 采用**页式内存管理**。
- 在原有 **4KB 物理页框分配器 (kalloc/kfree)** 与 **Sv39 页表**的基础上, 本项目重构并扩展了内存管理结构:
 - 物理页分配器 (kalloc) → **内核堆分配器 (kmalloc)** → 虚拟内存 (Sv39 页表) → **按需分页 (Lazy)** → **写时复制 (COW)** → **文件内存映射 (mmap)**



■ 虚拟内存管理

● 内核字节级堆分配器 (kmalloc/kmfree)

- **分配策略**: 采用 **First-Fit (首次适应) 算法**。
若空闲块过大则执行 Split (切分) 重新标记空闲; 无满足条件的空闲块时, 调用底层 kalloc 申请 4KB 新物理页挂入链表重新分配。
- **释放与合并**: **回收**内存块并**标记为空闲**, 随后扫描隐式空闲链表**合并物理地址相邻**的空闲块, 有效防止产生外部内存碎片。

```
struct kmem_header {
    uint64 size;           // 内存块大小 (不含首部, 单位: 字节)
    struct kmem_header *next; // 指向下一内存块 (隐式空闲链表指针)
    int is_free;           // 是否空闲 (1: 空闲, 0: 已分配)
};

// 全局堆管理器: 管理所有内核堆内存块
struct {
    struct spinlock lock; // 多核并发互斥锁
    struct kmem_header *head; // 隐式空闲链表头指针
} kmalloc_mem;
```

● 按需分页 (Lazy Allocation) <https://pdos.csail.mit.edu/6.S081/2020/labs/lazy.html>

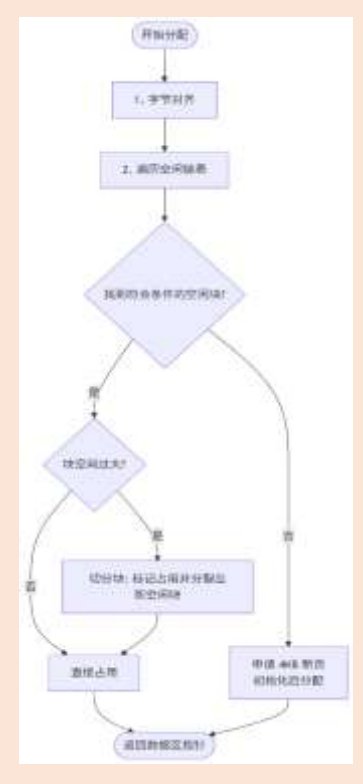
- **惰性分配**: 修改 sys_sbrk 系统调用, 增加进程虚拟地址空间时**仅抬高边界 p->sz**, **推迟物理内存分配**。
- **缺页补全**: 读写未映射地址时触发 scause 13/15 异常, 在 **usertrap()** 中**动态分配物理页**并利用 mappages 补齐映射, 实现对用户态的透明访问。

```
// sys_sbrk: 增长时仅抬高虚拟边界, 不分配物理页
uint64 sys_sbrk(void) {
    uint64 addr;
    int n;
    struct proc *p = myproc();

    argint(0, &n);
    addr = p->sz;

    if(n < 0) {
        if(-n > p->sz)
            return -1;
        // 内存缩减: 必须立即调用 growproc 释放物理页
        if(growproc(n) < 0)
            return -1;
    } else {
        // 延迟分配: 只向上增长虚拟边界 sz, 不分配物理页
        if(p->sz + n >= MAXVA || p->sz + n < p->sz)
            return -1;
        p->sz += n;
    }
    return addr;
}
```

```
// usertrap 中: 缺页异常动态装载
if(scause == 13 || scause == 15) {
    uint64 va = PGROUNDDOWN(r_stval());
    if(r_stval() < PGROUNDUP(p->sz)) { // 合法性校验 (含页内偏移)
        char *mem = kalloc(); // 分配物理页
        if(mem == 0) { setkilled(p); }
        else {
            memset(mem, 0, PGSIZE);
            if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, PTE_R|PTE_W|PTE_U) < 0) {
                kfree(mem);
                setkilled(p);
            }
        }
    } else {
        setkilled(p); // 非法越界地址, 结束进程
    }
}
```



■ 虚拟内存管理

● 写时复制 (Copy-On-Write Fork)

- **共享与引用计数**: fork 复制页表 (uvmcopy) 时**清除可写页写权限**, 打上 **PTE_COW** 标记并共享物理页。引入全局 **page_ref** 数组以自旋锁保护物理页引用计数。
- **物理页按需分裂**: 写入只读页触发异常, 检测到 **PTE_COW 且计数 > 1** 时, 分配新页并拷贝数据、更新映射, 递减原物理页引用计数; 若**计数 = 1** 则原地恢复写权限。

● 文件内存映射 (mmap/munmap)

- **虚拟区间登记**: 引入进程级 **struct vma** (最多支持 16 个槽位), 记录映射起始地址、长度、访问权限、映射标志及关联文件指针。
- **延迟读取与回写**: 实际访问触发**缺页**时通过 readi 读盘填充物理页。销毁区间时, 遍历已映射页, 对 **MAP_SHARED** 的脏页调用 **writei** 回刷磁盘。

```
// 全局物理页引用计数数组
struct {
    struct spinlock lock;           // 并发保护自旋锁
    char counts[PHYSTOP / PGSIZE]; // 物理页引用计数数组
} page_ref;

void ref_inc(uint64 pa) {
    if(pa < (uint64)0 || pa >= PHYSTOP)
        return;
    acquire(&page_ref.lock);
    page_ref.counts[pa / PGSIZE]++;
    release(&page_ref.lock);
}

void ref_dec(uint64 pa) {
    if(pa < (uint64)0 || pa >= PHYSTOP)
        return;
    acquire(&page_ref.lock);
    page_ref.counts[pa / PGSIZE]--;
    release(&page_ref.lock);
}

int ref_get(uint64 pa) {
    if(pa < (uint64)0 || pa >= PHYSTOP)
        return 0;
    int c;
    acquire(&page_ref.lock);
    c = page_ref.counts[pa / PGSIZE];
    release(&page_ref.lock);
    return c;
}
```



<https://pdos.csail.mit.edu/6.S081/2025/labs/cow.html>

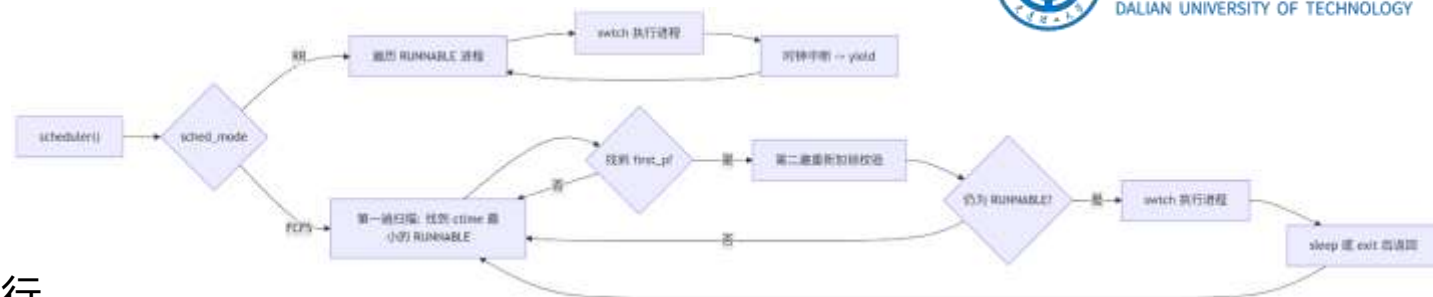
```
// 虚拟内存区域结构 (每个进程支持最大 16 个 VMA 槽位)
struct vma {
    int valid;           // 槽位是否被占用
    uint64 addr;         // 映射起始虚拟地址 (4KB 对齐)
    int len;             // 映射长度 (字节)
    int prot;            // 访问权限 (PROT_READ / PROT_WRITE)
    int flags;           // MAP_SHARED (共享回写) 或 MAP_PRIVATE (私有写时复制)
    struct file *f;      // 关联的文件结构体指针
    int offset;          // 映射在文件内部的偏移量
};
```

<https://pdos.csail.mit.edu/6.S081/2025/labs/mmap.html>

■ 进程管理

● NP-FCFS 与 RR 双调度器动态切换

- **模式切换**：通过全局调度模式变量 `sched_mode` 实现调度策略的动态切换。
- **RR 模式 (0)**：抢占式。时钟中断触发时执行 `yield()` 让出 CPU，重新进入就绪队列。
- **NP-FCFS 模式 (1)**：**非抢占式**。时钟中断不剥夺进程运行权，当前进程持续运行至主动 `exit()` 或 `sleep()` 阻塞。采用“**两遍扫描**”选取 **ctime 最小的就绪进程**，避免持锁释放间隙的竞态。



```
struct proc {
    // ...
    uint ctime;           // 进程创建时间戳: 用于 FCFS 调度选取最早进程
    enum procstate state; // 进程状态: UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
    // ...
}
```

```
struct proc *p = myproc();

acquire(&wait_lock);
for(;;){
    havekids = 0;
    for(pp = proc; pp < &proc[NPROC]; pp++){
        if(pp->parent == p){
            if(target_pid > 0 && pp->pid != target_pid) continue; // 匹配目标 PID
            acquire(&pp->lock);
            havekids = 1;
            if(pp->state == ZOMBIE){
                pid = pp->pid;
                int temp_xstate = pp->xstate;
                freeproc(pp); // 彻底回收资源
            }
        }
    }
    if(!havekids || killed(p)){ release(&wait_lock); return -1; }
    if(options == 1){ release(&wait_lock); return 0; } // WNOHANG
    sleep(p, &wait_lock); // 阻塞挂起等待
}
```

● 子进程精准回收与非阻塞 (waitpid)

- **精准回收**：若参数 `target_pid > 0`，则扫描子进程表时仅匹配 PID 对应的**特定**子进程；若 `target_pid == -1`，回退为回收**任意**子进程。
- **非阻塞 (WNOHANG)**：若指定 WNOHANG 且匹配的**目标子进程非 ZOMBIE 状态**，函数不挂起父进程，而是立即返回 0，**允许父进程继续执行**。

■ 进程管理

● 内核级同步信号量 Semaphore

- 与 xv6 的 sleep / wakeup 结合
- sleep(void *chan, struct spinlock *lk): 让当前进程在通道 chan 上睡眠。
- wakeup(void *chan): 唤醒所有在通道 chan 上睡眠的进程。
- 可以直接把信号量结构体本身的内存地址 s 作为睡眠通道 chan

- 基于**内核 sleep/wakeup**的底层睡眠锁机制。
- **sem_wait**: 持信号量自旋锁, **递减** count。若 count < 0, 调用内核 sleep(s, &s->lock), 将当前进程挂起在信号量**结构体内存地址** (睡眠通道) 上。
- **sem_signal**: 持锁**递增** count。若有进程在阻塞 (count <= 0), 调用 wakeup(s) **精准唤醒**在该**信号量通道**上睡眠的进程。

```
struct sem {
    struct spinlock lock; // 保护计数的自旋锁
    int count;             // 可用资源数
};

int sem_wait(uint64 sem_addr) {
    struct sem *s = (struct sem*)sem_addr;
    acquire(&s->lock);
    s->count--;
    while(s->count < 0) {
        sleep(s, &s->lock);
    }
    release(&s->lock);
    return 0;
}

int sem_signal(uint64 sem_addr) {
    struct sem *s = (struct sem*)sem_addr;
    acquire(&s->lock);
    s->count++;
    if(s->count <= 0) {
        wakeup(s);
    }
    release(&s->lock);
    return 0;
}
```

● Alarm 异步定时器机制 作用: 向用户进程提供**非阻塞的周期性异步事件通知机制** (类似 UNIX 信号 SIGALRM) 。

- **触发与跳转**: 中断计数达到阈值时, 通过 kmalloc 将当前 **trapframe 备份至 alarm_tf**, 修改 **epc** 强行跳转至用户处理函数。
- **防嵌套重入**: 处理函数执行期间设置 **alarm_running = 1**, 在此期间阻止定时器二次触发, 防止栈溢出。
- **现场还原 (sigreturn)**: 函数结束调用 sigreturn 系统调用, 将 **alarm_tf 覆盖还原回 trapframe** 并调用 kmfree 释放空间。

<https://pdos.csail.mit.edu/6.S081/2025/labs/traps.html>

```
struct proc {
    // ...
    int alarm_interval;           // 触发间隔
    uint64 alarm_handler;         // 用户态处理函数地址
    int alarm_ticks;              // 滴答累计计数
    struct trapframe *alarm_tf;   // 动态备份现场 (kmalloc 分配)
    int alarm_running;            // 标志位: 防处理函数重入触发
};
```

■ 文件系统

● lseek 文件定位

- 支持**进程**对普通**文件**的**随机读写与偏移指针定位**。
- 操作前调用 `ilock(f->ip)` 获取 Inode 级**睡眠锁**，保障在多进程/多核竞争下更新偏移量和文件大小一致。**过滤非法 fd 与非磁盘文件定位**。
- 支持 **SEEK_SET (0)**、**SEEK_CUR (1)**、**SEEK_END (2)** 模式。

```
if(f->type != FD_INODE)           // 仅支持普通 inode 文件
    return -1;

struct inode *ip = f->ip;
int new_off = f->off;

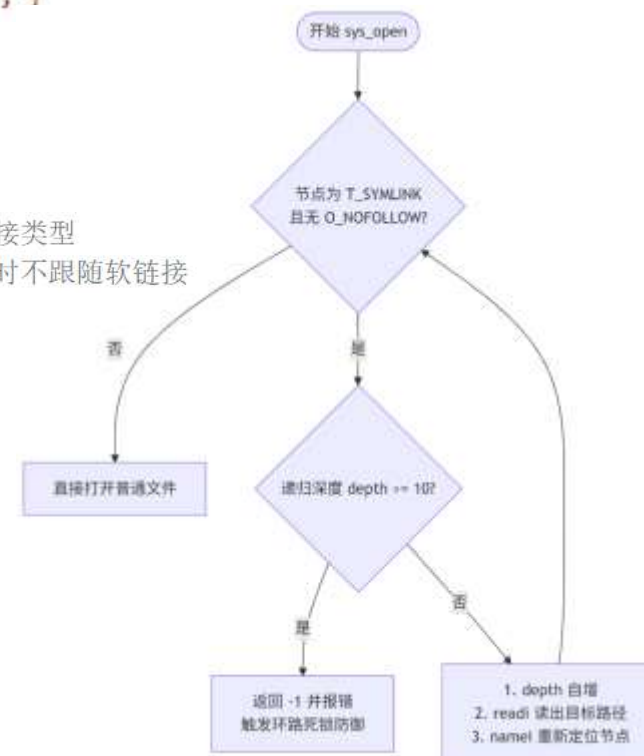
ilock(ip);                        // Inode 锁保护
if(whence == 0)    new_off = offset;
else if(whence == 1) new_off = f->off + offset;
else if(whence == 2) new_off = ip->size + offset;
else { iunlock(ip); return -1; }
```

● Symlink 软链接

核心作用：创建指向另一个目标文件路径的**特殊符号链接文件**。

- **创建软链接**：分配 **T_SYMLINK** 节点，利用 `writei` 将**目标路径字符串**直接写入其**数据块**中。
- **递归跟随解析**：在 `sys_open` 路径中检测到软链接时，利用 `readi` 读出**目标路径**，调用 `namei` **递归查找**。
- **环路检测保护**：限制**递归深度 depth 上限为 10**，超过则判定为环路死循环并中断报错，防止内核无限死循环。

```
#define T_SYMLINK 4           // 软链接类型
#define O_NOFOLLOW 0x004    // 打开时不跟随软链接
```



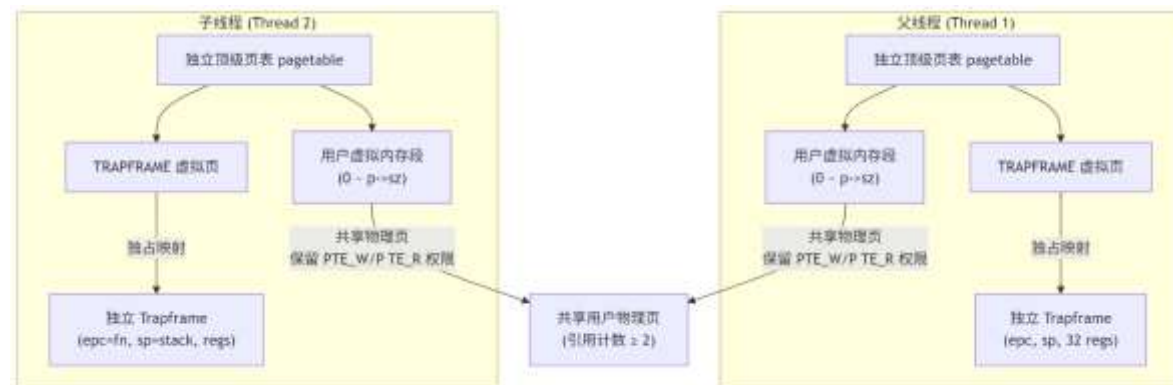
<https://pdos.csail.mit.edu/6.S081/2025/labs/fs.html>

■ 线程与同步机制

● clone 轻量级线程 (LWP 模型)

- **共享与隔离**: 为子线程分配**独立的 PCB 及专属的 trapframe 物理页**, 在 CPU 调度上下文切换时隔离核心寄存器。
- **物理共享**: 自定义 uvmsharecopy, 将**父进程的用户态虚拟页**表项原样 (不加 PTE_COW) 复制到**子线程的独立页表**中, 共享物理内存并调用 `ref_inc` 递增页计数。
- **状态拷贝与资源共享**:
 - **状态传递**: 子线程拥有**独立用户栈 stack**, 并在 trapframe 中通过 `a0` 传递参数。**线程组 ID 绑定为主进程 PID** (`tgid = p->tgid`) 。
 - **资源共享**: 通过 `filedup` 共享主进程的**文件表**和 **VMA 槽位**。
 - **独立退场**: 终结时**独立递减物理页计数**, 当线程组最后一个活跃成员退出时, 真正释放共享的虚拟空间物理页。

基于独立页表 + 物理页共享模型



```
struct proc {
    // ...
    pagetable_t pagetable;
    int is_thread;
    int tgid;
    // ...
};
```

// 独立顶级页表
// 线程标志位
// 线程组 ID (tgid == 主线程的 pid)

■ 线程与同步机制

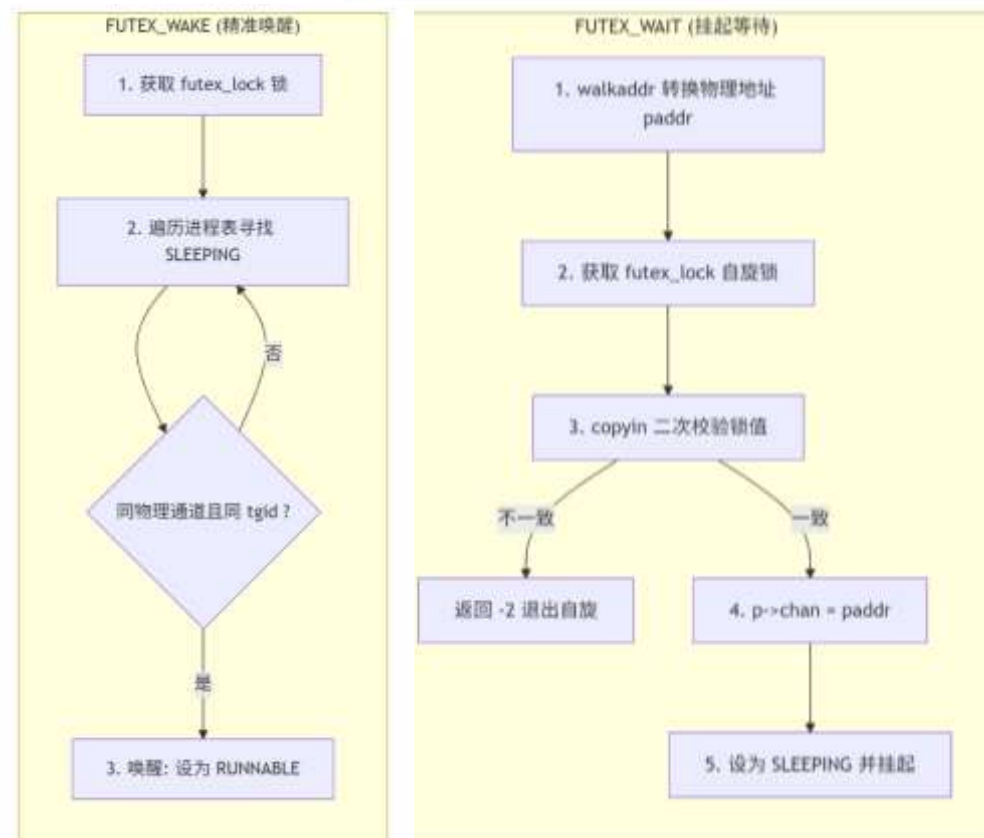
● Futex 用户态快速同步锁

线程间同步

- **核心设计：**无竞争时在用户态通过原子操作**快速加锁** (Fast-path)；面临**锁竞争**时，调用 sys_futex 系统调用**陷入内核**挂起或唤醒 (Slow-path)。
- **物理 Key 机制：**为避免进程间虚拟地址冲突，通过 walkaddr **提取锁的全局唯一物理地址 paddr**，作为内核精准休眠的通道标识 (chan)。
- **Lost-Wakeup 竞态消除：**在全局自旋锁 futex_lock 保护下，通过 **copyin 二次检查**锁值。若值已改变（说明锁已被快速释放）则退回用户态自旋，防止**产生检查与休眠之间的时序隐患**。
- **精准唤醒与组隔离：**遍历进程表，唤醒处于 **SLEEPING 状态**、**休眠通道等于目标 paddr**、且属于**同一线程组** (tgid 相同) 的线程，最多唤醒 val 个。

```
#define FUTEX_WAIT 0
#define FUTEX_WAKE 1
```

```
struct spinlock futex_lock; // 全局锁保护挂起原子性
```



■ 中断异常与用户程序

● 中断与异常处理 (Trap 机制)

- 异常分类分发 (右图)
- 异常防护与安全隔离: 若页面错误等异常无法修复, 内核**打印详细诊断信息**, 强制调用 `exit(-1)` **结束该损坏进程**, 保障内核**不发生 Panic**, 确保多核其他进程正常运行。

● 进程快照 ps

- 内核态采集 (`sys_getprocs`): 遍历全局进程表 `proc[]`, 对每个**非 UNUSED 状态的进程持锁读取**其 `pid`、`state`、`sz`、`name`, 存入内核临时缓冲区 `kprocs[]`。
- 安全拷贝至用户态: 通过 `copyout()` 将内核缓冲区中的进程快照数组**安全写入用户态提供的内存地址 `uaddr`**, 由页表机制自动处理 COW/Lazy 缺页。
- 用户态格式化输出 (`ps.c`): 用户程序调用 `getprocs()` **获取快照数组**, 按列格式化输出 PID、STATE、SIZE、NAME。

```

用户态运行 → 中断/异常 → trampoline.S → usertrap()
├── scause == 8 → 系统调用 → syscall() 分发
├── devintr != 0 → 硬件中断 (时钟/键盘/磁盘)
│   └── which_dev == 2 → 时钟中断 → alarm 检测
└── 其他异常 → 分类拦截
    ├── scause 2 → 非法指令 → 打印诊断 → Kill 隔离
    ├── scause 13 → 读段错误 (Load Page Fault)
    │   ├── VMA 区间 → mmap 按需调页
    │   └── 堆区间 → Lazy Allocation
    ├── scause 15 → 写段错误 (Store Page Fault)
    │   ├── VMA 区间 → mmap 按需调页
    │   ├── PTE_COW → COW 分裂
    │   └── 堆区间 → Lazy Allocation
    └── 其他 → 打印诊断信息 → setkilled(p)
    
```

```

struct uproc {
    int pid;           // 进程 ID
    int state;         // 进程状态 (UNUSED / USED / SLEEPING / RUNNABLE / RUNNING / ZOMBIE)
    uint64 sz;         // 虚拟地址空间大小 (字节)
    char name[16];     // 进程名称
};
    
```

```
yuans@yuans-virtual-machine:~/os/xv6-riscv$ make qemu
```

```
xv6 kernel is booting
```

```

hart 2 starting
hart 1 starting
hart 3 starting
init: starting sh
$ ps
PID  STATE  SIZE  NAME
1    SLEEPING 16384  init
2    SLEEPING 20480  sh
3    RUNNING 16384  ps
$
    
```

■ 系统测试与稳定性验证

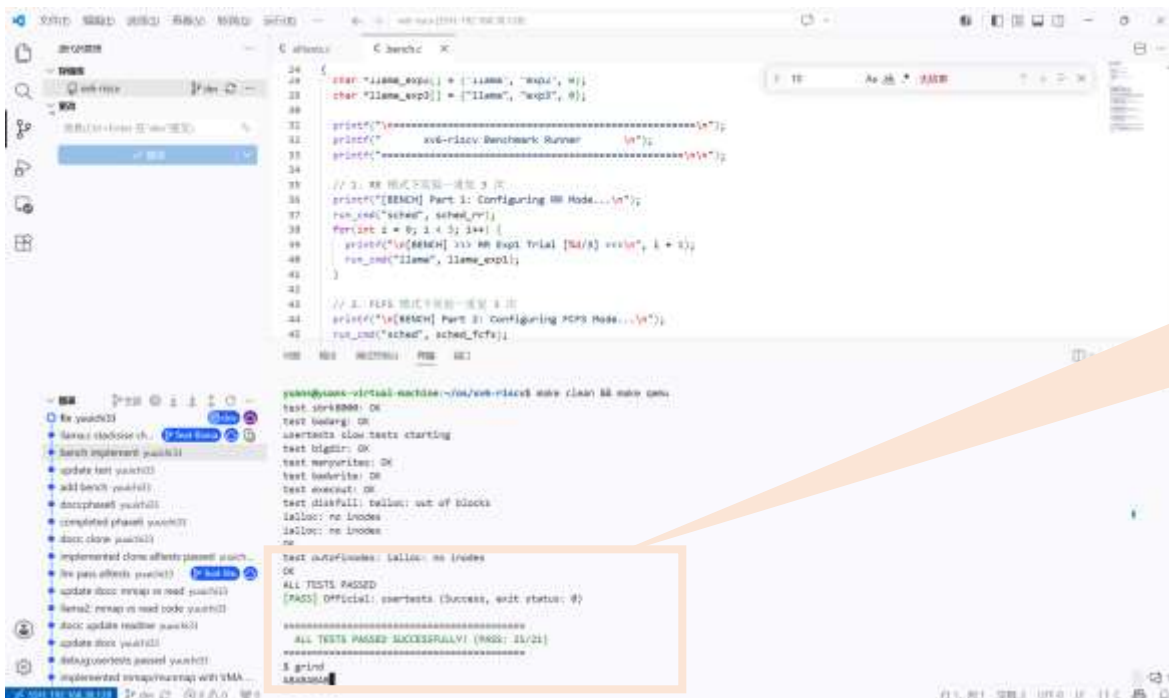
● alltests.c: 统一测试框架

<https://github.com/mit-pdos/xv6-riscv-fall19/>

- **20 项增量单元测试**: crash_test.c、ps.c、kmalloctest.c、sched.c、schedtest.c、waitpidtest.c、semtest.c、alarmtest.c、symlinktest.c、lazytests.c、cowtest.c、mmaptest.c、clonetest.c、futextest.c。
- **xv6 原生 usertests**: 系统调用参数合法性、进程与内存管理、文件系统功能、并发与压力、异常与鲁棒性。

● grind.c: xv6 原生压力测试

- 创建**两个子进程**，高强度**随机执行 23 种操作** (fork、kill、文件读写、sbrk、管道等)，在**持续运行**过程中未出现 panic、死锁或异常退出。字符交替输出 (如 `ABBABA`) 表明多核同步锁设计正确。



- **结论**: 运行 alltests 全量通过 (`PASS: 21/21`) ; grind 连续运行**数十分钟**，系统**稳定**。

■ 量化性能评估与对比实验 – LLM 推理引擎移植与性能量化

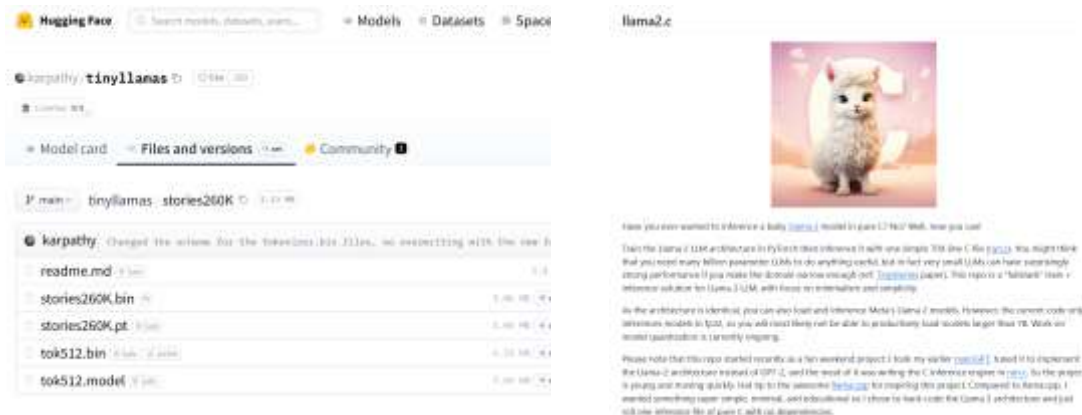
● 研究背景:

- **大语言模型**在边缘设备运行时, 需要**多核并发**、**线程同步**以及**文件映射**等内核机制的支持。
- **原生 xv6-riscv** 偏向教学设计, 原有的**多并发性能和磁盘 I/O 效率较弱**, 难以直接运行此类程序。

● 测试目的: 通过移植一个实际程序, **定量测试**本项目增量实现的 clone、futex、mmap **在实际运行中的真实表现**。

◆ 基准程序 (llama.c) 的选择依据:

- 选用 Andrej Karpathy 开源的**极简 Transformer 推理程序 llama2.c**, 加载 **1.04MB** 的 **stories260K 模型**。
- 该程序的计算主要集中在**矩阵-向量乘法** (matmul), 占总计算时间的 90% 以上, **适合用于测试多核心调度与线程同步机制**。



<https://github.com/karpathy/llama2.c>

◆ 具体适配与移植工作:

- **数值逼近实现数学函数**: 由于 xv6 用户态不支持 **<math.h>**, 因此采用泰勒级数等数值逼近算法, 手实现了推理所需的 exp、sqrt、sin、cos 函数。
- **静态工作线程池**: 利用新增的 clone 构建静态线程池以**替代原版的 OpenMP**。并分别实现了 Spinlock (空转)、Pipe (管道)、Futex (快速锁) 三种同步模式。
- **文件读取路径重构**: 将**文件操作**重构为**底层系统调用**, 实现 mmap 映射模式与 malloc+read 传统模式的对比。

● 自动化测试脚本: bench.c

注: 此处, 实验一与二有概率发生卡死现象, 初步定位该问题与 sys_futex 的慢速路径调度时序有关, 或其他原因, 仍需进一步分析。

■ 量化性能评估与对比实验 – 实验1：多核并行计算实验

- **实验目的：**将矩阵乘法（matmul）算子切分到多个核心上并发执行，验证**多线程计算在多核下的实际运行时间改善**。
- **测试配置：**
 - QEMU 模拟 **4 物理核心**（-smp 4），工作**线程数分别设置为 1、2、4** 运行。
 - 使用 **Futex** 作为同步机制，分别测试 **RR 与 FCFS** 两种调度策略。
- **计算密度放大（FACTOR）：**
 - 由于测试模型较小（stories260K **隐藏层维度仅 64**），单次矩阵计算开销极小。为了避免内核上下文切换开销掩盖并行计算效果，在矩阵算子内层循环引入了**计算密度放大因子**（FACTOR = 100）。

调度模式	线程数	轮 1	轮 2	轮 3	平均 Ticks	加速比
RR	1	36	39	35	36.7	1.00×
	2	23	25	24	24.0	1.53×
	4	20	20	20	20.0	1.84×
FCFS	1	35	35	36	35.3	1.00×
	2	24	25	25	24.7	1.43×
	4	21	22	22	21.7	1.63×

- **加速效果：**1 线程增加到 2 线程时运行时间减少明显（RR 模式下加速比 1.53×），表明行切分负载**能够较好利用多核**。
- **边际递减：**从 2 线程到 4 线程时加速比变慢（最高 1.84×），**符合 Amdahl 定律**（非并行计算部分和内核自旋锁等同步开销限制了线性提升）。
- **调度器对比：**抢占式 RR 表现略好于非抢占 FCFS，主要是频繁睡眠/唤醒时，RR 的及时抢占机制能让就绪的 worker 线程更快获得 CPU。

■ 量化性能评估与对比实验 – 实验2：同步机制对比实验

- **实验目的：**量化对比**自旋锁**、**内核管道**、**Futex** 三种同步机制在多线程频繁同步时的**效率损耗**。
- **测试配置：**
 - RR 调度模式下配置 **4 个工作线程**，重复测试 3 轮，每轮推理生成 **10 个 Token**。
- **三种同步机制的差异：**
 - **Spinlock（自旋锁）：**在用户态对原子共享标记变量进行忙等待，不执行系统调用。
 - **Pipe（内核管道）：**工作线程通过 read() 系统调用阻塞，主线程通过 write() 写入同步信号唤醒。
 - **Futex（快速锁）：**工作线程通过 futex_wait 进入内核挂起，主线程完成计算后通过 futex_wake 精准唤醒。

同步方式	轮 1	轮 2	轮 3	平均 Ticks	相比 Spinlock
Spinlock	270	266	269	268.3	-
Pipe	36	33	32	33.7	快 87.4%
Futex	20	20	21	20.3	快 92.4%

- **Spinlock 的效率瓶颈：**由于物理 CPU 仅有 4 个，但计算线程达到了 5 个。自旋锁导致无任务的线程死死占用物理核心而不主动让出，造成了**严重的 CPU 空转**浪费（耗时达 **268.3** Ticks）。
- **Pipe 的挂起让出：**使用 Pipe 后，**等待线程**调用 **read()** **立即阻塞并被移出就绪队列**，将物理核心让渡给其他线程，运行效率大幅提升（**33.7** Ticks）。
- **Futex 的效率优势：**Futex 相比 Pipe 在平均时间上进一步缩短了 **40%**（仅需 **20.3** Ticks）。主要原因在于其只有在真正需要睡眠时才陷入内核，省去了部分**不必要的系统调用及上下文切换开销**。

■ 量化性能评估与对比实验 – 实验3：存储映射冷启动对比实验

- **实验目的：**验证并对比大模型权重文件在 **malloc + read** 阻塞载入与 **mmap** 文件存储映射机制下的**冷启动初始化耗时**。
- **测试配置：**
 - **单线程**运行，针对 1.04MB 规模的 stories260K.bin 模型权重文件，分别调用两种不同的读取方式。
 - 用 **uptime()** 系统调用测量加载前后的 Ticks 差值作为冷启动耗时
- **两种加载方式的机制区别：**
 - **malloc + read 模式：**通过 sbrk 申请内存空间，再通过标准系统调用进行阻塞式读盘，伴随**内核到用户态的双重数据拷贝**。
 - **mmap 模式：**调用 sys_mmap 在页表中**建立文件的虚拟 VMA**，实现**延迟装载**。

加载方式	I/O 机制	冷启动耗时
malloc + read	用户堆空间申请 + 阻塞式磁盘读取 + 内核/用户态双重拷贝	21 Ticks
mmap	仅页表虚拟 VMA 建立 + 后续读写触发按需缺页调入（零拷贝）	0 Ticks

- **mmap** 的冷启动延迟为 **0 Ticks**，对比 **malloc + read** 的 **21 Ticks**，在首屏加载速度上有数量级优势。按需调页机制将磁盘 I/O 分散到推理过程中，避免了启动时的阻塞等待。

■ 创新点、不足及展望

● 创新点

- **完整的内存管理层次**：本系统的内存子系统在 Sv39 页表及物理页框分配器的基础上，自下而上构建了**结构完整、层次分明的虚拟内存体系**：物理页分配器 (kalloc) → **内核堆分配器 (kmalloc)** → 虚拟内存 (Sv39 页表) → **按需分页 (Lazy)** → **写时复制 (COW)** → **文件内存映射 (mmap)**。
- **轻量级线程与用户态快速同步**：基于**独立顶级页表 + 物理共享实现 clone 线程** (LWP)，配合 **futex 用户态快速锁**为并发编程提供了完整的基础设施。在 llama2.c 推理程序中验证了多线程并行+高效同步的实际效果。
- **多维度测试验证与应用闭环**：系统的**稳定性**在**功能** (alltests)、**压力** (grind)、**真实重载性能** (bench) **三个层面得到验证**。将 **LLM 推理引擎作为系统的真实负载**，在逻辑上闭环验证了各系统调用在边缘状态下的健壮度，同时为操作系统的优化方向提供了可量化的客观参考。

● 不足及展望

- **同步调度开销**：FCFS 在两阶段扫描释放锁期间存在同步窗口，拟引入核心本地就绪队列减少全局锁竞争。
- **文件描述符共享差异**：目前 clone 采用 filedup 复制文件表而非完全共享，与 POSIX 线程标准存在差异。
- **偶发性挂起问题**：少数情况下运行实验存在偶发卡死，初步分析与 sys_futex 调度时序有关，需进一步分析并修复。
- 当前 kmalloc/kfree 基于 First-Fit 算法，可进一步实现更高效的 Buddy System 或 Slab Allocator。
- 实现 ProcFS 虚拟文件系统。
- 完善用户态 libc
-

■ 参考资料（部分）

- <https://github.com/mit-pdos/xv6-riscv>
- <https://github.com/mit-pdos/xv6-riscv-book/>
- <https://github.com/qemu/qemu>
- <https://github.com/sandyyyzy/Re-XVapor>
- <https://github.com/tianx666/xv6-book-riscv-rev1-Chinese>
- https://blog.csdn.net/zzy980511/category_11740137.html
- <https://github.com/mit-pdos/xv6-riscv-fall19>
- <https://github.com/torvalds/linux>
- <https://pdos.csail.mit.edu/6.S081>
- <https://linux-kernel-labs.github.io/>
- <https://pdos.csail.mit.edu/6.S081/2020/labs/lazy.html>
- <https://fail.lingfei.xyz/tags/xv6/>
- <https://github.com/karpathy/llama2.c>

■ 附录：测试录屏

- <https://pan.baidu.com/s/1f9yu-8JZlu-6vaPEfcVp3Q?pwd=1s23>



大连理工大学

DALIAN UNIVERSITY OF TECHNOLOGY

感谢！ 敬请批评指正

汇报人：袁善

2026-06-22